

SMART LOG ANALYZER & ANOMALY DETECTOR

Technical Assessment • Duration: 3.5 Hours

Problem Statement

Systems generate large volumes of log data, and spotting unusual or problematic events manually is slow and error-prone. Build an application that ingests log data, flags unusual entries using an algorithm of your choice, and uses AI to explain what was flagged and why.

Basic Requirements

Build an application that loads and persists a set of log entries (at minimum a timestamp, event type, severity, and source), flags unusual entries using an approach of your own choosing, and persists each flagged entry along with a brief reason or score explaining why it was flagged. Use AI to turn each flagged entry into a plain-English explanation and a likely root cause or next step. The application should let a user view and manage the logs — for example, a list or timeline with flagged entries highlighted, plus a detail view — and should handle basic validation issues such as missing timestamps, malformed entries, or an empty dataset.

Sample Data (Example Only)

The lines below are a small illustrative example of what a log entry could look like — they are not a dataset to use as-is. You are expected to synthesize your own dataset (mostly-normal entries with a handful of injected anomalies works well). It does not need to follow this exact structure or column set, as long as it captures comparable log information such as a timestamp, source/IP, event or request type, and a status/severity indicator.

Timestamp	IP Address	Status	Message
2026-08-20 09:14:02	192.168.1.14	200	GET /api/users – success
2026-08-20 09:14:05	192.168.1.14	404	GET /api/orders/9981 – not found
2026-08-20 09:15:10	10.0.0.55	500	POST /api/payment – internal server error
2026-08-20 09:15:45	203.0.113.7	403	GET /admin – access denied
2026-08-20 09:16:02	192.168.1.14	200	GET /api/users – success

You can refer to this dataset as well, just for reference: <https://github.com/Anchit-Nayak/Assignment-Data>

AI Requirement

The detection of unusual entries should be your own implementation — don't have the AI decide what counts as anomalous. The AI component should be used specifically to generate the plain-English explanation and likely root cause / next step for each entry your implementation has already flagged — not to simulate this with hard-coded or templated text.

Your Approach

Tech stack, architecture, and schema are not prescribed — make your own calls.

Optional Enhancements

Brownie points: add any functionality of your own choosing — original, well-justified additions are viewed favorably.

Submission

Source code + README (setup, AI config, approach, assumptions, limitations).

Assessment Focus

Area	Focus
Functionality	Working and coherent solution
AI	Appropriate use, reasoning and reliability
Engineering	Design, code quality and maintainability
Data	Persistence and sensible data handling
UX	Clarity and usability
Problem solving	Technical decisions, trade-offs and originality

AI coding assistants may be used. You must be able to explain your implementation and decisions.